LAB7

Data Structure

Midterm Exam

- **Wednesday, April 22, from 11:00 AM to 1:00 PM**.
- Students whose student ID numbers end in an **even digit (0, 2, 4, 6, 8)** : **IT/BT Room 501**.
- Students whose student ID numbers end in an **odd digit (1, 3, 5, 7, 9)** : **IT/BT Room 510**.
- Please make sure to check your assigned exam room in advance.
- For identity verifiaction, please bring a **student ID card**.

Lab 7

- The deadline for lab6 submission is 4/16 THU at 11:59 pm.
- If you have any questions, please contact TA through email. (112alsdud@naver.com)
- Lab scores will be uploaded to the LMS next week.
- Questions about Lab 7 scores will only be accepted during next lab session.

- Code name: {student_id}_p7.c
- The code will be tested with 5 different input files.
- Each input file will be worth 20 points
- Make sure that the output format matches the provided example exactly.

Evaluation criteria

Category	Evaluation	
p7	100	
Total	100	

- *Use GCC **13.3** version.*
- *No score will be given if there is a compile error caused by difference of gcc version*

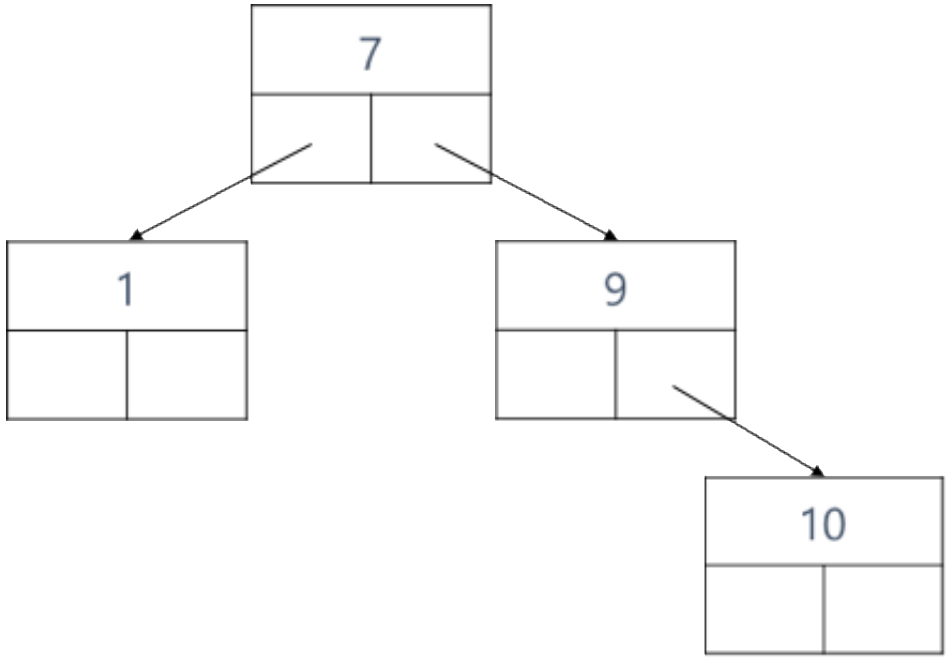
Lab 7 – Binary Search Tree

- Design and implement a Binary Search Tree (BST) using a linked list.
- Use complete binary tree ADT to perform pre-, in-, and post-order traversal using recursive functions.
- In addition to the traversals, your program must also:
  - **Insert** a node into the BST.
  - **Delete** a node from the BST.
  - **Find** the **minimum** and **maximum** node.
  - **Find** a **specific node** by key.
  - **Compute the height** of the BST.

Lab 7 – Binary Search Tree

- Structure

```
typedef struct BinarySearchTree* Tree;
struct BinarySearchTree
{
        int value;
        Tree left;
        Tree right;
};
```



Lab 7 – Binary Search Tree

- Structure

```
typedef struct BinarySearchTree* Tree;
struct BinarySearchTree
{
        int value;
        Tree left;
        Tree right;
};
```

- Function

```
<Lab7>
Tree insertNode(Tree root, int key);
Tree deleteNode(Tree root, int key);
Tree findMin(Tree root);
Tree findMax(Tree root);
void findNode(Tree root, int key);
int getHeight(Tree root);
void printPreorder(Tree root);
void printInorder(Tree root);
void printPostorder(Tree root);
```

Lab 7 – Binary Search Tree

- **Tree insertNode(Tree root, int key)**
  - Insert a node with the specified key into the BST and print "Insert [key]"
  - If the key already exists in the tree, print an error message; "Insertion Error: There is already [key] in the tree"
- **Tree deleteNode(Tree root, int key)**
  - Delete a node with the specified key from the BST and print "Delete [key]"
  - Reform tree **using left subtree**.
  - If the key does not exist in the tree, print an error message; "Deletion Error: [key] is not in the tree"
- **Tree findMin(Tree node)**
  - Return the node with the minimum key value and print "Min: [key]"
- **Tree findMax(Tree node)**
  - Return the node with the maximum key value and print "Max: [key]"

Lab 7 – Binary Search Tree

- **int getHeight(Tree root)**
  - Compute and return the height of the BST and print "Height: [height]"
  - The height of leaf nodes is 0.
- **void findNode(Tree root, int key)**
  - Search for a node with the given key.
  - Print '[key] is in the tree' if the key exists. Otherwise, print '[key] is not in the tree'
- **void PrintInorder(Tree root)**
  - Print the BST using in-order traversal.
- **void PrintPreorder(Tree root)**
  - Print the BST using pre-order traversal.
- **void PrintPostorder(Tree root)**
  - Print the BST using post-order traversal.

Lab 7 – Binary Search Tree

- **Input Cases**
  - **i x** insert a new key “x” into the binary search tree without duplication.
  - **d x** delete a key “x” in the binary search tree.
  - **mi** find the minimum key value in the binary search tree.
  - **ma** find the maximum key value in the binary search tree.
  - **f x** find the given key to check whether the key exists in the tree.
  - **h** compute the height of the binary search tree.
  - **pr** print the tree by pre-order traversal.
  - **pi** print the tree by in-order traversal.
  - **po** print the tree by post-order traversal.

Lab 7 – Binary Search Tree

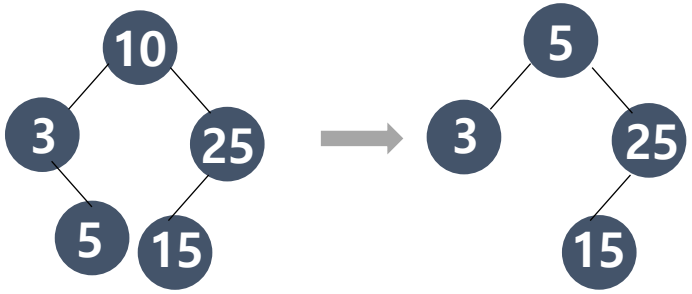
```c
#include<stdio.h>
#include<stdlib.h>

typedef struct BinarySearchTree* Tree;
struct BinarySearchTree {
    int value;
    Tree left;
    Tree right;
};

Tree insertNode(Tree root, int key);
Tree deleteNode(Tree root, int key);
Tree findMin(Tree root);
Tree findMax(Tree root);
int getHeight(Tree root);
void findNode(Tree root, int key);
void printInorder(Tree root);
void printPreorder(Tree root);
void printPostorder(Tree root);
```

```c
int main(int argc, char* argv[]) {
    FILE *fi = fopen(argv[1], "r");
    char cmd;
    int key;

    Tree root = NULL;

    while(!feof(fi)) {
        fscanf(fi, "%c", &cmd);
        switch(cmd){
        case 'i':
            fscanf(fi, "%d", &key);
            root = insertNode(root, key);
            break;
        case 'd':
            fscanf(fi, "%d", &key);
            root = deleteNode(root, key);
            break;
        case 'p':
            fscanf(fi, "%c", &cmd);
            if(cmd == 'i')
                printInorder(root);
            else if(cmd == 'r')
                printPreorder(root);
            else if(cmd == 'o')
                printPostorder(root);
            printf("\n");
            break;
        case 'f':
            fscanf(fi, "%d", &key);
            findNode(root, key);
            break;
        case 'm':
            fscanf(fi, "%c", &cmd);
            if (cmd == 'i') {
                Tree minNode = findMin(root);
                if (minNode) printf("Min: %d\n", minNode->value);
            } else if (cmd == 'a') {
                Tree maxNode = findMax(root);
                if (maxNode) printf("Max: %d\n", maxNode->value);
            }
            break;
        case 'h':
            printf("Height: %d\n", getHeight(root));
            break;
        default:
            break;
        }
    }
    return 0;
}
```

Lab 7 – Binary Search Tree

- input file : lab7_input.txt

```
i 10
i 25
i 3
i 5
i 15
i 10
mi
ma
pr
pi
po
f 30
f 5
h
d 10
h
po
d 20
h
pr
```



- Result

```
Insert 10
Insert 25
Insert 3
Insert 5
Insert 15
Insertion Error: There is already 10 in the tree
Min: 3
Max: 25
10 3 5 25 15
3 5 10 15 25
5 3 15 25 10
30 is not in the tree
5 is in the tree
Height: 2
Delete 10
Height: 2
3 15 25 5
Deletion Error: 20 is not in the tree
Height: 2
5 3 25 15
```